

Andrej Makemore L2

Brain Dump:

- Taking a dataset of names [t*t]
- ~~embed~~ numerical representations → sorted-index for set(words)
"join"
↓
alphabet.
a-z
- creating <S> & <E>
- discarding it take cause its ineff for counting bigram occurrences
↳ count of two char in the dataset.
- having . instead.
S → counts
↓
end
S → start
- because <E> a
"bc.. never occurs"
a <S> never occurs

→ Did all of this using tensor datatype.

Then:

Salata

Part: Sampling from methods pre req:

Deterministic: given the same input & conditions, you always get the same output.

↳ to do this use seed for generating random numbers.

Multinomial: samples indices from a probability distribution

replacement = True

↓
can have duplicate values
1, 1, 1, ...

given [0.6, 0.2, 0.2]

samples: 20 10

output:

1 1 1 1 1 1 2 2 3 3
60% 20% 20%

Pseudocode:

for - in range(20)
ix = 0 → element → start, end.

while True: first possible char

P = N[ix], float()

P = P / P.sum → prob.

ix = torch.multinomial(P, num_samples=1, replace=True, generator=g).item()

most likely
[not the highest]

And sample a probability.

out.append(tos[i*x])
if ix == 0 → returned back to - send here.
break.

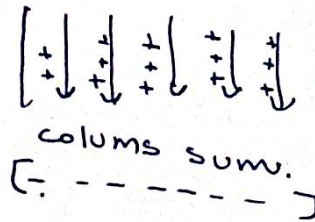
print(out. " ". join(out))

Section: efficiency! vectorized normalization of rows..

sum(input, ~~int~~ dim, keepdims=T/F)

P. Sum(0) → keepdims=T
→ row axis

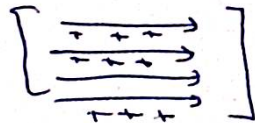
returns [1, 27]
dim



P. sum(0) keepdims=F)
returns [27]

→ squeeze generates 2d representation

To sum row wise
go thru the columns
-----→



then P-sum(1, keepdims=T) → [27, 1] → col vector $\begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}$
(in bigram that we cons, the sum row col & row are identical)

Broadcasting rules:

if [27, 27]
[27, 1]

→ it stretches it out [copies it 27 times column wise] element wise. [27, 27]

Rule: 2 dim sizes tensors are broadcastable if

• Starting at the trailing dimension, the dim sizes must either be equal or one of them is 1 or one of them does not exist. ③

eg: $\begin{pmatrix} 5 & 3 & 4 & 1 \\ 1 & 3 & 1 & 1 \end{pmatrix}$

Rules: ③ ① ② ② ✓

$\begin{pmatrix} 2 & 4 & 1 \\ 3 & 1 & 1 \end{pmatrix}$
X ② ②

→ NOT Broadcastable.

0.1 0.3 0.4 0.9
a b c d
b
c
dth 0.1 0.4 0.3 0.2
a b c d e

de
ethaw 0.1 0.2 0.7 0.3
a b c d

dep
0.9 0.2 0.3
a b c d e
def.

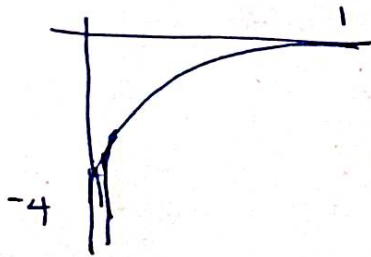
Likelihood:

- loss function.
- how well our model performs
- it is the product of all individual probabilities assigned by model to the correct next characters.

$$L(\theta|x) = \prod_{i=1}^n f(x_i; \theta)$$

Log likelihood → ~~prob~~ ^{cause} multiplication of probabⁿ (0-1) might yield tiny values.

Log:



prob
→ 0 → 1 → extends towards $-\infty$
as it goes toward 0 → sharp -ve.

multi
x b → adds $\log(a) + \log(b)$
↓

stable & easy to compute

Loss func: - log likelihood
minimize error.

→ log likelihood gives -ve values
since (0-1)
• we flip it to +ve no.

For a loss:

0 → should be the best possible score

* For probability 1 to the next correct character
then $\log(1) = 0$

then $-\log(1) = 0$

We need to bring the ~~the~~ likelihood of the entire data towards 0.

Another example:
→ probability of next char high.

$$\log(0.9)$$

$$\log(0.2)$$

then $+\log(0.9) = -0.0458$

then $+\log(0.2) = -0.699$

then $-\log(0.9) = 0.0458$
good.

then $-\log(0.2) = 0.699$
bad

here 0.0458 is closer to 0 than ↑

Python Run
here we will take the avg. of all likelihood

Eg: .emma

. e . e m m m m a

↓

input to neural net = 0 [index of .]

↳ ① one hot [1.00(27)]

② W → logits

③ exp, ~~log~~ counts. → probabilities
Soft max.

output of neural net;

[0.6 , 0.10, 0.012 ... till 27]

label → $y_s(\hat{e}) \rightarrow 5$

probab : 0.012

↓

compute log likelihood

$\log(0.012)$

↓

-4.399

neg log:

4.399 [high loss]

Do the same for em, mm, ma.

4.399, 4.014, 2.623, 2.60, 4.20

Likelihood → sum and avg neg log likelihood is

$$\frac{4.399 + 4.014 + 2.623 + 2.6 + 4.2}{4} \rightarrow \text{loss} = 3.769$$

[high loss]

Building a neural net:

Step 1 take indices

↓
one hot encoding

↓
plug w into these.

↓
log counts → logits

exponentiate

Soft max.

find probability

$$\frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

→ output of that index

[1 0 0 0]

→ [1 0 0 0]

lookup-table

@

w_1	w_2	w_3	w_4
w_{21}	w_{22}	w_{23}	w_{24}
w_{31}	w_{32}	w_{33}	w_{34}
w_{41}	w_{42}	w_{43}	w_{44}

→ gives embedding for character a
(plucks out w for a)

eventually becomes a learned representation after training

then:

[1 0 0 0]

a b c d

→ [1 0 0 0]

next character

@

	a	b	c	d
a	0.2	-0.4	0.7	0.1
b	-0.5	0.8	-0.2	0.3
c	0.6	0.1	-0.3	0.9
d	0.4	-0.6	0.2	0.5

scores: logits

next character
[0.2 -0.4 0.7 0.1]
a b c d

exp:

So next character is c

ac

↑

c has the highest probability of 40 %

behave like true counts
Since logits have -ve vals.

← [1.22 0.67 2.01 1.11]

Normalize (prob)

[1.22 / 6.7 0.67 / 6.7 2.01 / 6.7 1.11 / 6.7]
0.67 + 1.22 + 2.01 + 1.11

← [0.24 0.13 0.40 0.22]
29%

Summary:

Implemented 2 bigram models.

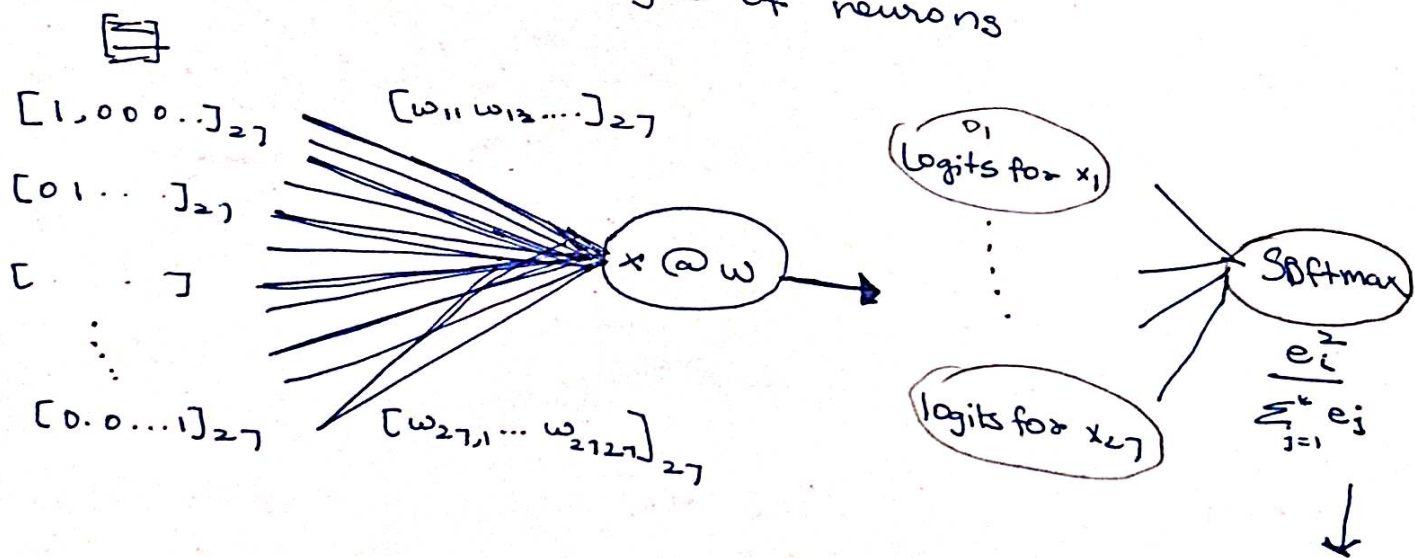
1st way:

frequency:

- build a frequency matrix to store bigram counts
↓
normalized to create a probability distribution.
↓
sampled new names.

2nd way: Neural networks

- Used a single linear layer of neurons



- logits $\rightarrow$ log-counts $\rightarrow$ exp $\rightarrow$ prob.
- Optimization using pyTorch [always flush the gradient before updating]

• Regularization.

penalize large weights - preventing the model from being over confident.

$W^2 \cdot \text{mean}()$

Laplace smoothing:

add 1 to counts ~~even~~ even if it didn't occur [-inf problem]